WEBPROGRAMOZÁS – BEADANDÓ DOKUMENTÁCIÓ

Magyarországi foglalkoztatási és lakossági statisztikai webalkalmazás

Tantárgy: Web-programozás 1

www: HTTPS://bravehearts.hu/webprog
Github: https://github.com/pugneri/webprog_eloadas

A dokumentáció a projekt célját, architektúráját, adatbázisát, felületeit, telepítési lépéseit és minőségi szempontjait ismerteti.

Készítette: Pugner István - MS9N6H & Ujvárosi Mihály - GJJSEO

Dátum: 2026

Tartalomjegyzék

1. Bevezetés 3
2. Célok 4
3. Könyvtárszerkezet és fájlok szerepe 5
4. Adatbázislogika és relációk 6
5. Konfiguráció és szerveroldali architektúra 7
6. Főoldal, navigáció és közös stílusréteg 8
7. JavaScript CRUD modul részletes leírása 9
8. React CRUD modul részletes leírása 10
9. SPA modul részletes leírása 11
10. Fetch API modul részletes leírása 12
11. Axios modul részletes leírása 13
12. OOJS modul részletes leírása 14
13. UI-finomhangolás, toastok és validáció 15
14. Buildelés, Node/npm környezet és hibakezelés 16
15. GitHub, release és csapatmunka 17
16. Tesztelési forgatókönyvek 18
17. Követelmények megfeleltetése 19
18. Továbbfejlesztési lehetőségek 20
19. Összegzés 21
Beadás előtti ellenőrzőlista 22

1. Bevezetés

A beadandó feladat célja egy olyan webalkalmazás elkészítése volt, amely egyszerre mutatja be a klasszikus kliensoldali JavaScript, a komponensalapú React-fejlesztés, a szerveroldali adatbázis-kommunikáció, valamint az objektumorientált JavaScript-programozás gyakorlatát. A megoldás közös témája a magyarországi régiók, megyék, lakosságszámok és foglalkoztatási adatok feldolgozása lett, mert ez a témakör jól strukturálható relációs adatbázisra, ugyanakkor kellően szemléletes ahhoz, hogy több különböző nézetben is értelmes felhasználói felületet lehessen rá építeni.

A rendszer tervezésekor az volt a fő szempont, hogy ne hét teljesen különálló felület jöjjön létre, hanem egy olyan, egységes világra épülő alkalmazás, ahol minden oldal ugyanarra a domainre támaszkodik. A felhasználó így nem elszigetelt technológiai példákat lát, hanem ugyanannak az adathalmaznak eltérő megközelítéseit: egyszerű tömbalapú kezelést, React állapotkezelést, egylapos interakciót, illetve backenddel kommunikáló modulokat.

A dokumentáció ennek megfelelően nem pusztán felsorolja az elkészült fájlokat, hanem értelmezi is a projektet. Célja, hogy a beadandó minden fontos része visszakövethető legyen: milyen adattáblákra épül, hol találhatók a kliensoldali és szerveroldali állományok, hogyan működik a konfigurációs réteg, miként történik a build-elés, és milyen tesztelési lépések szükségesek ahhoz, hogy a rendszer stabilnak tekinthető legyen.

2. Célok

A feladat értelmezésénél fontos volt felismerni, hogy a követelmény nem csupán egy technológiai lista kipipálását jelenti. A főoldal, a JavaScript CRUD, a React CRUD, a SPA, a Fetch API, az Axios és az OOJS rész együtt alkotják a beadandót. Ebből következik, hogy a projekt minőségét nem kizárólag az dönti el, hogy minden menüpont működik-e, hanem az is, hogy ezek mennyire alkotnak egységes rendszert. A projekt célja ezért az lett, hogy egyetlen közös adatvilágot több módszerrel lehessen kezelni és bemutatni.

A választott téma azért bizonyult jó döntésnek, mert a régiók és megyék viszonya jól modellezhető, a lakossági és foglalkoztatási adatok pedig számszerűsített, jól táblázható információk. Ez lehetővé tette, hogy a felhasználói felületek ne csak listákat mutassanak, hanem interakciókat, számításokat, játékos elemeket és grafikus kártyamegjelenítést is kapjanak.

A projektcél tehát három rétegben foglalható össze. Az első a funkcionális réteg: legyenek működő CRUD felületek és interaktív oldalak. A második az architekturális réteg: a rendszer legyen jól szervezett, különválasztott könyvtárszerkezettel és konfigurációval. A harmadik a minőségi réteg: a felület legyen áttekinthető, a hibakezelés legyen kulturált, a dokumentáció pedig legyen elég részletes ahhoz, hogy önmagában is bemutassa a fejlesztés gondolatmenetét.

3. Könyvtárszerkezet és fájlok szerepe

A projektkönyvtár tudatos felépítése fontos része volt a munkának. A gyökérben találhatók azok a HTML állományok, amelyeket a felhasználó közvetlenül megnyithat. Ezek a belépési pontok: index.html, javascript.html, react.html, spa.html, fetchapi.html, axios.html és oojs.html. Ezek mind külön oldalként működnek, de közös menürendszer és stílusvilág köti össze őket.

A közös erőforrásokat az assets mappa tartalmazza. Ide kerültek a közös CSS szabályok, a common.js jellegű segédfájlok, illetve azok a sima JavaScript állományok, amelyek nem React környezetben futnak. Ezzel párhuzamosan a React alapú megoldások külön alprojektet kaptak: a react-crud, a spa-app és az axios-react mappákban saját src és dist könyvtárak találhatók. A src tartalmazza a forráskódot, a dist pedig a buildelt állapotot, amelyet a gyökér HTML oldalai betöltenek.

A szerveroldali PHP-végpontok az api mappában kaptak helyet. A konfiguráció elkülönítésére létrejött az include mappa, amely a config.inc.php fájlt tartalmazza. Ez a strukturális elválasztás egyrészt átláthatóbbá teszi a projektet, másrészt csökkenti annak esélyét, hogy ugyanaz a konfigurációs részlet több helyen eltérően szerepeljen.

4. Adatbázislogika és relációk

A rendszer relációs adatbázis-szemléletre épül. A legegyszerűbb törzsadat a régió, amelyhez név és azonosító tartozik. A megyék a következő logikai szintet jelentik: minden megye egy régióhoz tartozik, vagyis a megyek tábla a regiok tábla azonosítójára hivatkozik. Ez az 1:N kapcsolat biztosítja, hogy a regionális bontás egyértelműen kezelhető legyen.

A lakossági adatokat a lakosok tábla tárolja, amely megyei azonosító és lakosságszám párosokkal dolgozik. A foglalkoztatási adatokat a foglalkozasok tábla tárolja, ahol egy technikai azonosító mellett mezőgazdasági, ipari és szolgáltatási mezők találhatók. A választott modell előnye, hogy a különböző információtípusok nincsenek egyetlen nagy, nehezen karbantartható táblába ömlesztve.

Az adatbázis-tervezésnél a legfontosabb követelmény az volt, hogy a későbbi frontendek számára egyaránt kiszolgálható legyen egyszerű lista, összekapcsolt nézet és szerkeszthető rekordkészlet. Ezért a relációk nem pusztán elméleti szerkezetet adnak, hanem közvetlenül támogatják a JOIN lekérdezéseket, a lenyíló választók feltöltését és a felhasználóbarát megjelenítést is.

Tábla	Mezők	Kapcsolat	Funkció
regiok	regiokod, regionev	1:N	Régiók törzsadatai.
megyek	megyekod, regiokod, megyenev	N:1 a regiok felé	Megyék azonosítása és régióhoz rendelése.
lakosok	megyekod, lakosszam	kapcsolat a megyek felé	Lakossági adatok tárolása.
foglalkozasok	fkod, megyekod, mezogazdasag, ipar, szolgaltatas	kapcsolat a megyek felé	Foglalkoztatási bontás tárolása.

A relációk lehetővé teszik, hogy a felhasználói felületeken kódok helyett megye- és régiónevek jelenjenek meg, miközben a háttérben továbbra is a kulcsmezők biztosítják az adatbázis konzisztenciáját.

5. Konfiguráció és szerveroldali architektúra

A szerveroldali réteg egyik legfontosabb minőségi javítása az volt, hogy az adatbázis-kapcsolat nem maradt szétszórva az API-fájlokban. Ehelyett létrejött az include/config.inc.php állomány, amely központilag tartalmazza a host, adatbázisnév, felhasználónév és jelszó adatokat, illetve felépíti a PDO kapcsolatot. Az api mappában lévő PHP fájlok ezt include vagy require_once segítségével töltik be.

Ennek a megoldásnak több előnye van. Egyrészt csökkenti az ismétlődő kód mennyiségét, másrészt megkönnyíti a karbantartást. Ha a projekt helyi fejlesztői környezetből tárhelyre kerül, akkor elegendő egyetlen konfigurációs fájlt módosítani. Harmadrészt világosabbá válik a projekt biztonsági modellje is: a tényleges elérési adatok elkülönülnek az üzleti logikától.

Az API-végpontok feladata, hogy jól definiált JSON választ adjanak minden kérésre. A frontend szempontjából különösen hasznos, ha minden válasz tartalmaz egy success jelzőt, egy értelmes message mezőt, illetve szükség esetén a data tartalmat. Ez egységesíti a Fetch API és Axios oldalak működését, és egyszerűbbé teszi a hibaüzenetek, illetve toast visszajelzések kezelését.

6. Főoldal, navigáció és közös stílusréteg

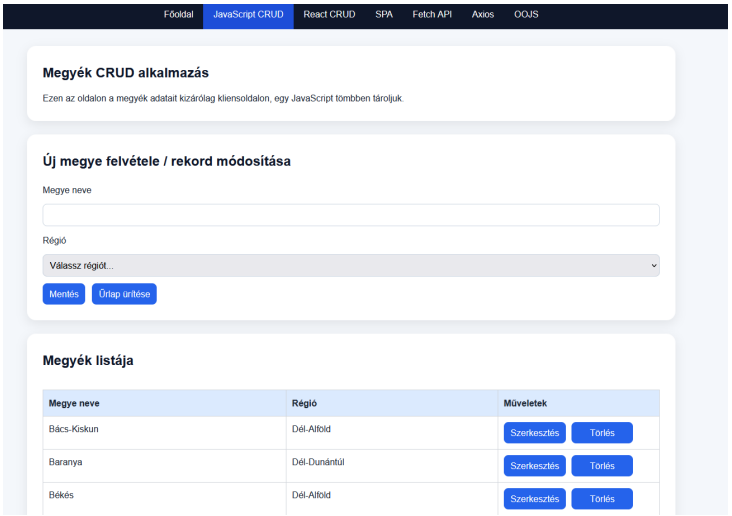
A főoldal a rendszer vizuális belépési pontja. Itt jelenik meg a projekt címe, rövid leírása és a menürendszer, amelyből az összes további modul elérhető. Bár a főoldal funkcionálisan egyszerűbb, mint a CRUD és API oldalak, mégis jelentős szerepe van, mert itt alakul ki a felhasználó első benyomása az alkalmazásról. Ezért fontos, hogy a címadás, a tipográfia, a kártyák és a menü jól strukturált legyen.

A közös stílusréteg célja, hogy a különböző technológiákkal készült oldalak mégis egy rendszer részeinek tűnjenek. Ennek része a színpaletta, a fejléc és a menü egységes megjelenése, továbbá a táblázatok, kártyák és gombok alapvető tipográfiai és térköz-szabályai. A fejlesztés során több esetben kiderült, hogy a vizuális minőséghez nem elég a „nagy” szerkezet: a gombok közötti távolság, a sorok szellőssége és az igazítások legalább ilyen fontosak.

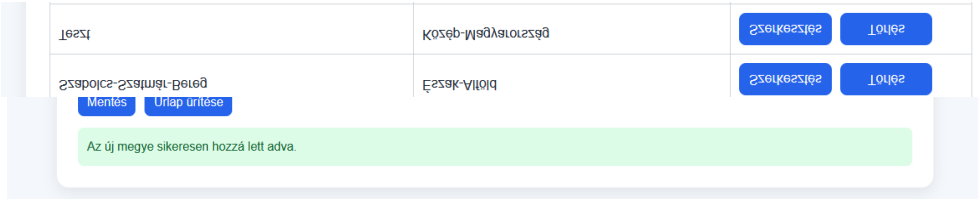
A common.js feladata többek között az aktív menüpont vizuális kijelölése. Ez apróságnak tűnik, de nagyban javítja a használhatóságot: a felhasználó mindig látja, hogy éppen melyik nézetben dolgozik. A közös UI-elvek ilyen kis elemekből állnak össze.

7. JavaScript CRUD modul részletes leírása

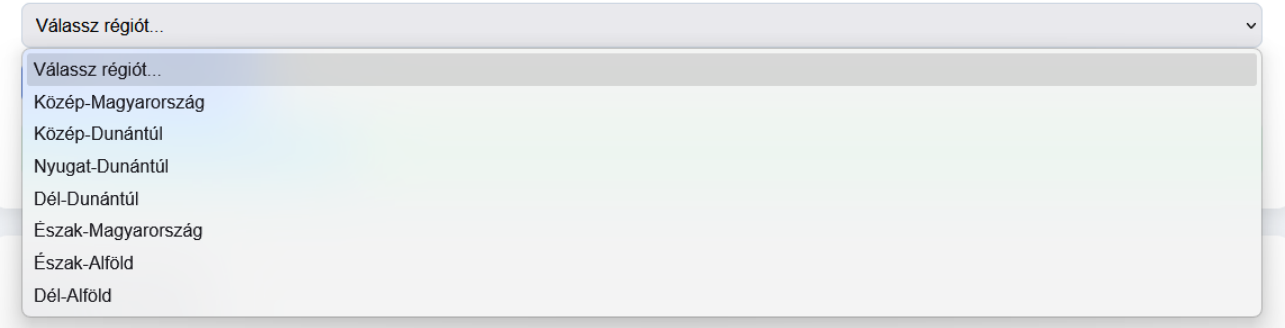
A JavaScript CRUD modul tisztán kliensoldali megközelítés. Itt a rekordok nem adatbázisban, hanem JavaScript tömbökben élnek, ezért a felület újratöltése után a módosítások nem maradnak meg. A modul pedagógiai szerepe azonban éppen ez: világosan megmutatja, hogyan működik a CRUD logika akkor, amikor a DOM-manipulációt és az állapotot teljes egészében a böngészőben kezeljük.



A modul a megyék adatain keresztül mutatja be a működést. A felhasználó új megyét vehet fel, módosíthatja a meglévő adatokat és törölhet elemeket. A rendszer a megjelenítést minden művelet után újrarendereli, így jól követhető, hogyan alakul át az oldal állapota. Ez különösen hasznos azoknak a részeknek az értelmezéséhez, ahol később ugyanezt a logikát már React vagy szerveroldali kommunikáció segítségével oldjuk meg.

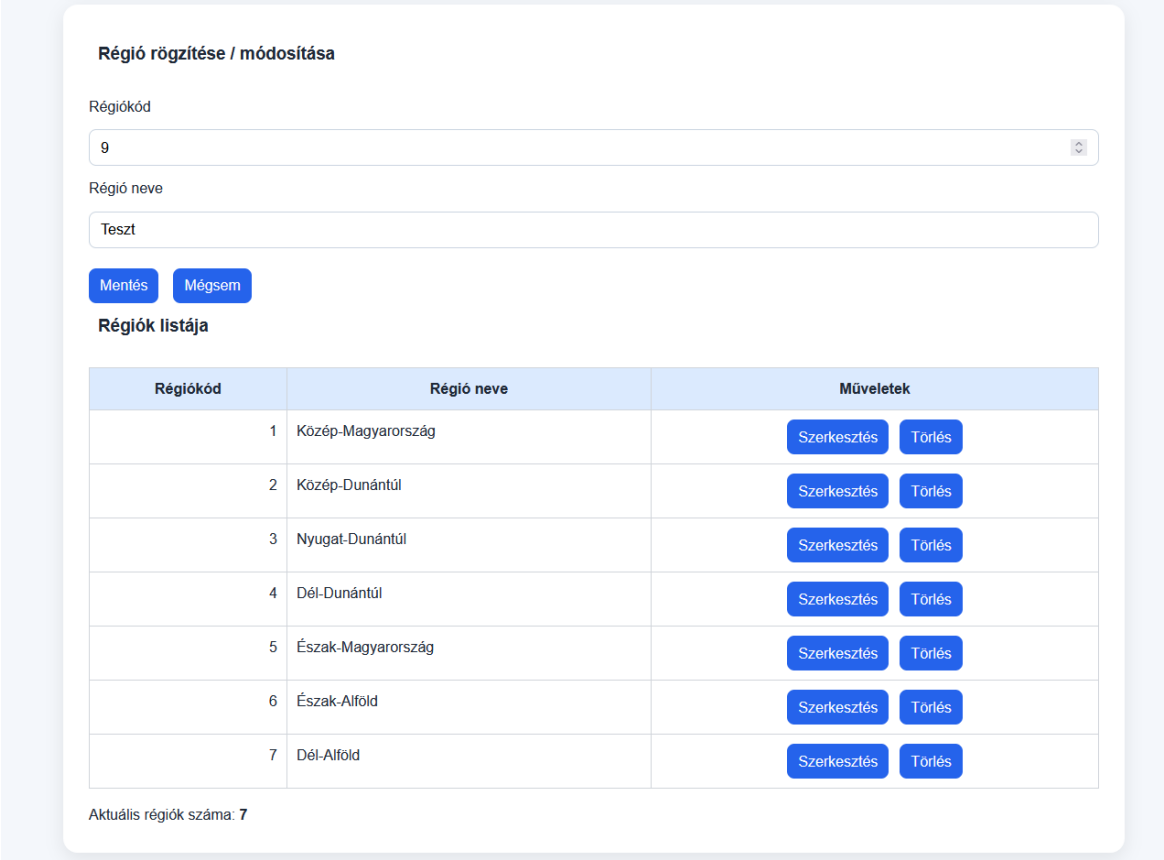


A későbbi finomhangolás során a nyers kódok helyett cél volt az emberbarát megjelenítés: ahol lehet, lenyíló választót és értelmesebb címkézést kapott a felület. Ezzel a modul nemcsak oktatási példává vált, hanem a teljes rendszer egyik, vizuálisan is vállalható komponensévé.



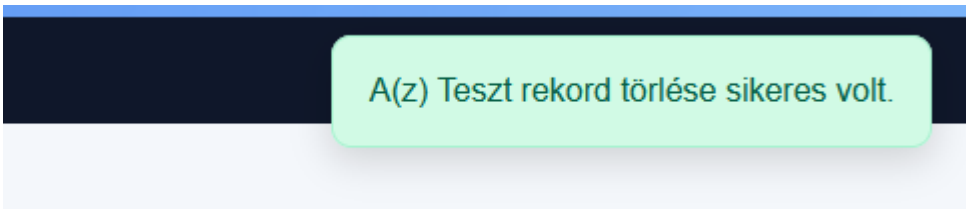
8. React CRUD modul részletes leírása

A React CRUD modul a régiók adatainak kezelését valósítja meg useState-alapú állapotkezeléssel. A régiólista és az űrlapmezők külön állapotokban élnek, ami jól elkülöníti a megjelenítést és az adatkezelést. Új elem létrehozásakor a lista bővül, szerkesztéskor egy meglévő rekord cserélődik le, törléskor pedig a megfelelő elem kikerül a tömbből.



Reactes szemléletben a fő előny az, hogy a komponens állapota az egyetlen „forrásigazság”, és a felület ebből épül fel. Ez olvashatóbbá teszi a logikát, mint a direkt DOM-írás, és előkészíti az utat a bonyolultabb, például API-ból töltődő modulokhoz is. Ugyanakkor a projekt során az is kiderült, hogy a stílusok és a buildelt assetek kezelése több odafigyelést igényel, mint egyszerű statikus oldalak esetében.

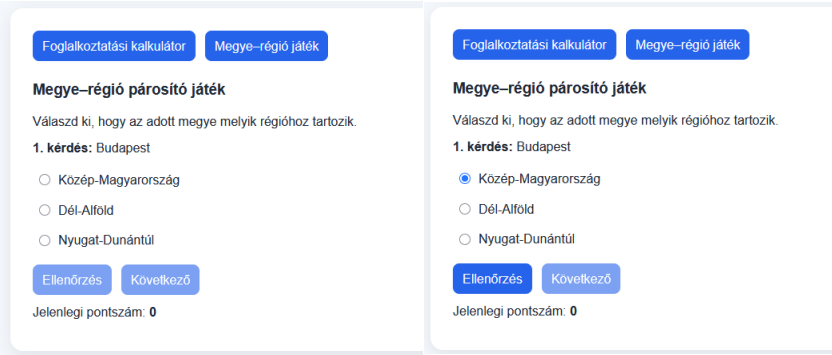
A modulhoz kapcsolódó felhasználói visszajelzések később toast-alapú irányba fejlődtek. A success, error és warning megkülönböztetés nemcsak esztétikai kérdés, hanem használhatósági is: másként kell kinéznie egy sikeres mentésnek, egy validációs hibának és egy semleges tájékoztatásnak.



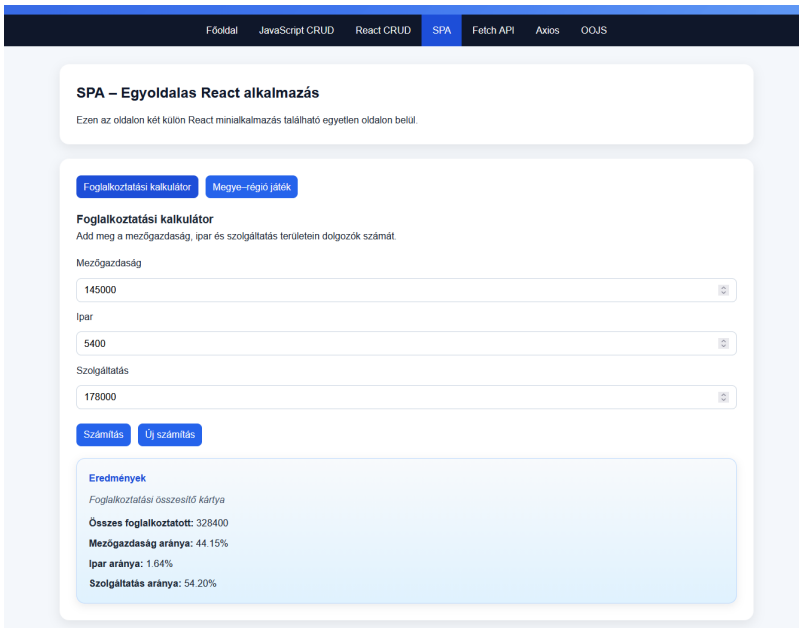
9. SPA modul részletes leírása

A SPA rész két mini alkalmazást egyesít egyetlen oldalon belül. Az első rész, a foglalkoztatási kalkulátor, a három gazdasági szektor értékeiből számol összesítést és százalékos arányt. A második rész, a megye–régió játék, tartalmi tudást kér számon és visszajelzést ad a felhasználó válaszáról. A két miniapp azért jó választás, mert egyszerre mutat be számítási logikát és játékos, állapotfüggő interakciót.

A SPA játékban különösen fontos volt a gombok közti állapotlogika. Az Ellenőrzés gomb csak akkor aktív, ha a felhasználó választott opciót, a Következő gomb pedig csak akkor, ha az aktuális kérdés ellenőrzése megtörtént. Ez a viselkedés biztosítja, hogy a felhasználó ne ugorhassa át a kérdéseket érvénytelen állapotban. A rádiógombok és a visszajelző mező közötti térközök, valamint az eredménykártya vizuális kialakítása sok finomhangoláson ment keresztül, mert a SPA esetében a kis UI-részletek erősen befolyásolják az élményt.



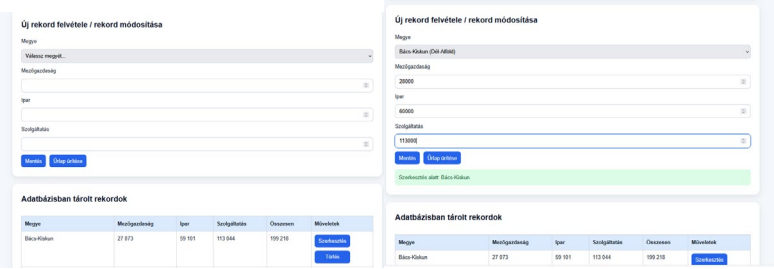
A kalkulátor eredményblokkja végül kártyás megjelenést kapott, hogy illeszkedjen az OOJS-modul vizuális nyelvéhez. Ez mutatja, hogy a projekt különálló részei között nemcsak funkcionális, hanem design-szintű kapcsolat is létrejött.



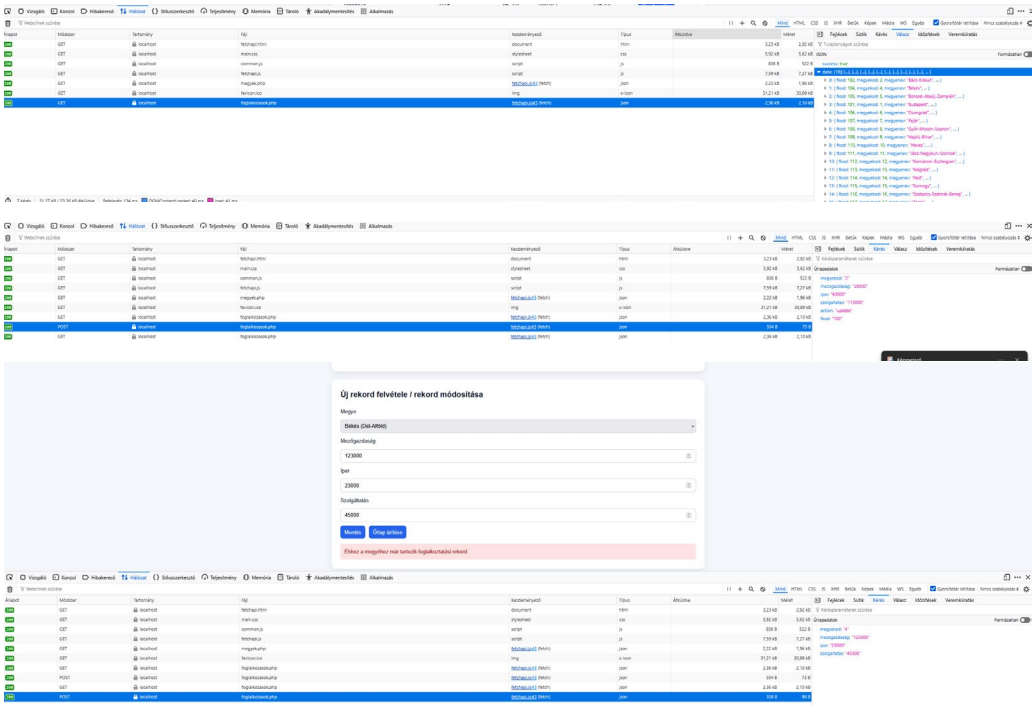
10. Fetch API modul részletes leírása

A Fetch API modulban a kliensoldali JavaScript már valódi szerveroldali adatokkal dolgozik. A modul a foglalkoztatási rekordok listázását, felvitelét, módosítását és törlését valósítja meg. Az egyik fontos fejlesztési cél itt az volt, hogy a megyekód helyett a megye neve jelenjen meg, és a felhasználó legördülő listából választhassa ki a megyét. Ez a döntés javítja az adatbevitel minőségét és olvashatóbbá teszi a listát.

A modul fejlesztése során felmerült, hogy a tiszta PUT és DELETE módszerek bizonyos környezetekben nem működnek megbízhatóan. Emiatt egyes megoldásokban POST alapú akciójelölésekre kellett támaszkodni. Ez jól mutatja, hogy az elméletileg „szebb” HTTP-séma és a gyakorlatban stabil megoldás nem mindig ugyanaz. A beadandó szempontjából az a fontos, hogy a műveletek megbízhatóan működjenek a választott környezetben.

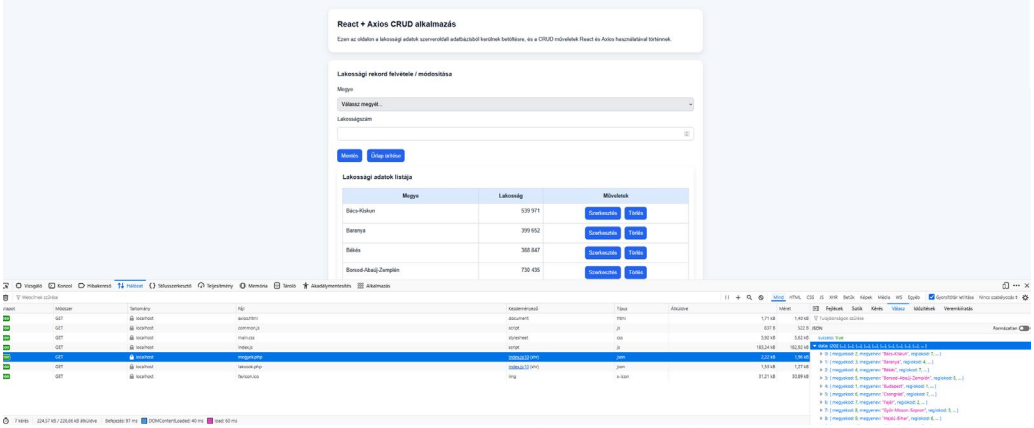


A Fetch API oldal így nemcsak technológiai bemutató, hanem egyben jó példa arra is, hogyan kell a felhasználóbarát felületet és a szerveroldali megbízhatóságot összehangolni.

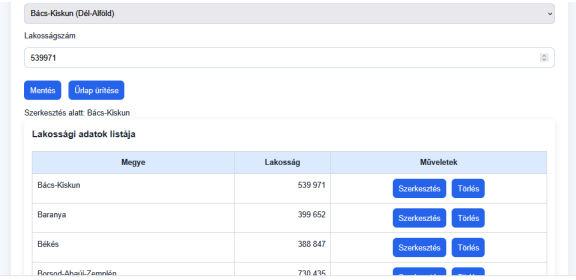


11. Axios modul részletes leírása

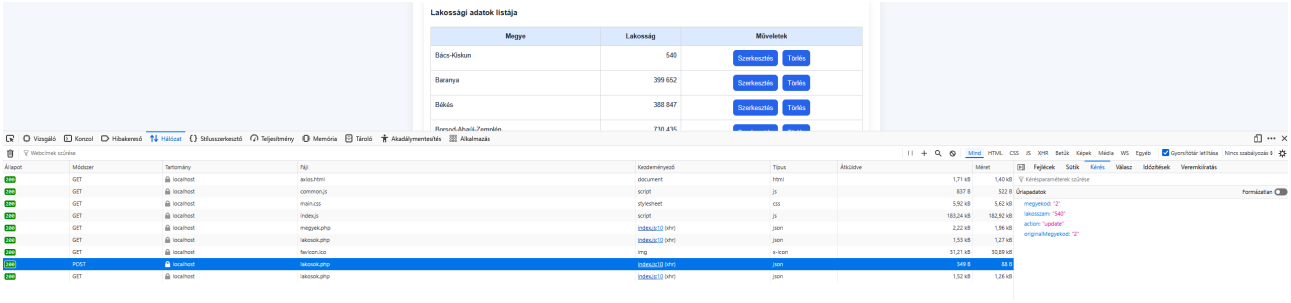
Az Axios modul a lakossági adatok kezelésére koncentrál. Az oldal React komponensben jeleníti meg az adatokat, Axios segítségével kommunikál a backenddel, és a válaszok alapján frissíti a táblázatot. A felület kialakításánál külön hangsúly került arra, hogy a lakosságszám oszlop jobbra igazítva jelenjen meg, mert a számok így könnyebben összehasonlíthatók. A műveleti gombok középre igazítása szintén ezt a rendezett, táblázati logikát erősíti.



A megye kiválasztása itt is lenyíló listából történik. Ez csökkenti a felhasználói hibák számát, és egyben igazodik ahhoz az alapelvhez, hogy a technikai kódok helyett lehetőség szerint értelmes, olvasható adatokat kell a felületen megjeleníteni. Az oszlopok közötti levegő, a cella padding és a gombok közti távolság többszöri finomhangolás után nyerte el végleges formáját.



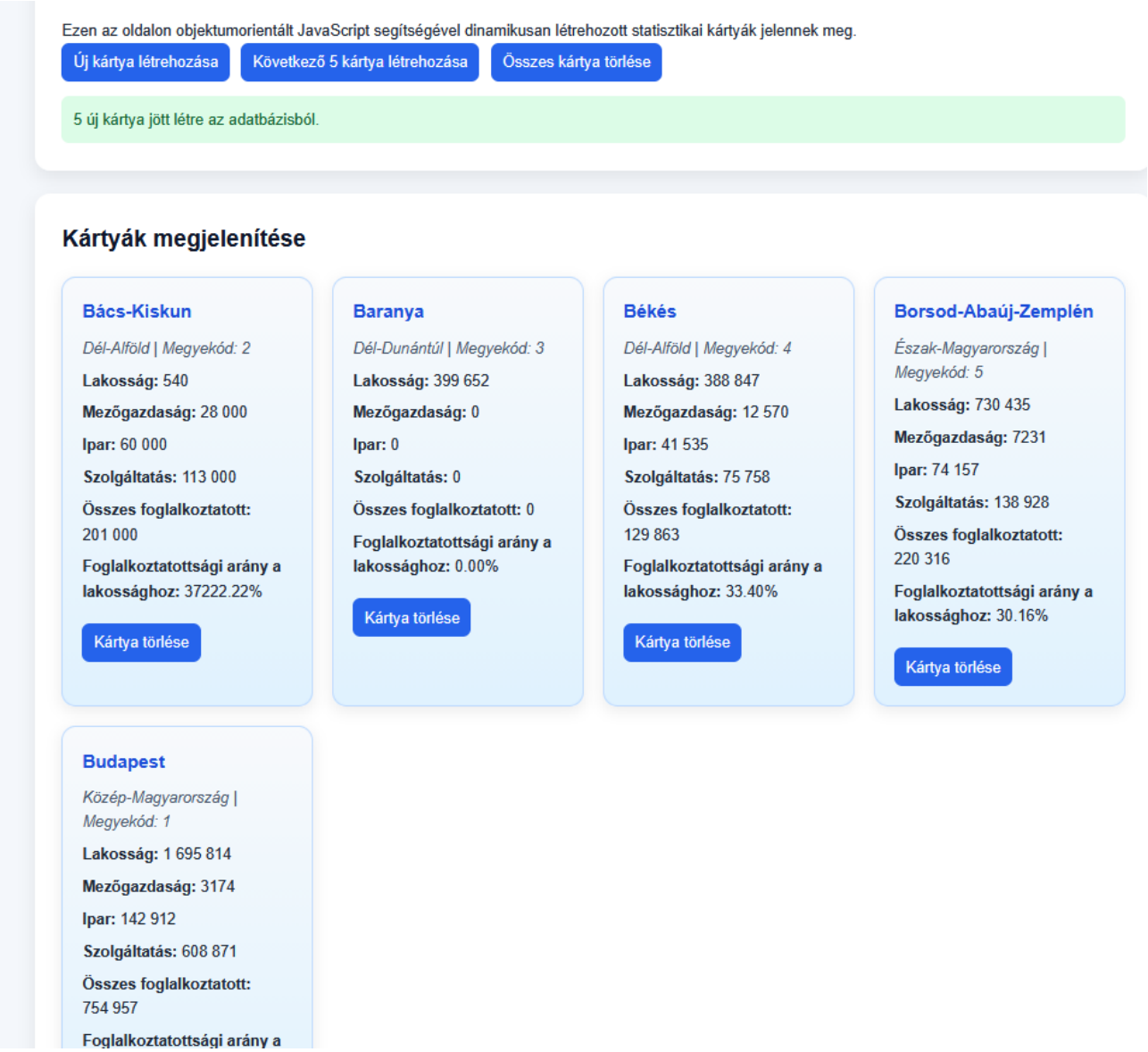
A modul jól példázza, hogy a React + Axios párosítás akkor ad igazán kulturált felhasználói élményt, ha a backend válaszstruktúrája konzisztens, a kliensoldali állapotkezelés pedig világosan szétválasztja a betöltés, siker, hiba és szerkesztés állapotait.



12. OOJS modul részletes leírása

Az OOJS rész feladata egy objektumorientált szemléletű grafikus megoldás elkészítése volt. A rendszer statisztikai kártyákat hoz létre, amelyek megyei adatokhoz kapcsolódnak. A megoldás osztályokat használ, konstruktorokat, saját metódusokat, valamint öröklődést. Ezek a fogalmak nemcsak formálisan vannak jelen a kódban, hanem valós szerepet kapnak a DOM-elemek létrehozásában és karbantartásában.

A fejlesztés során láthatóvá vált az OOJS szemlélet előnye: a kártya mint komponens viselkedése jól izolálható. A cím, az aladatok, a törlés logikája és a megjelenési séma egyetlen osztályban vagy kapcsolódó osztályhierarchiában tartható össze. Ez jól olvasható és bővíthető szerkezetet eredményez.



A kezdeti mintaadatos megoldás később adatbázis- vagy API-orientált irányba mozdult el, ami megmutatta, hogy az OOJS nem csupán oktatási játék, hanem valódi adatmegjelenítési minta is lehet.

13. UI-finomhangolás, toastok és validáció

A projekt hosszú része szólt a vizuális finomhangolásról. A gombok közti távolságok, a rádiógombok közötti sorközök, a táblázati cellák paddingja, a számok igazítása és a műveleti oszlopok elrendezése mind olyan pontok voltak, amelyek elsőre apróságnak tűnnek, de valójában a használhatóság és a beadandó minőség szempontjából kulcsfontosságúak.

A toast rendszer bevezetése a nyers, statikus üzenetdobozok helyett modernebb felhasználói élményt adott. A sikeres mentés zöld, a hiba piros, az információ vagy figyelmeztetés sárga színvilágot kapott. Ezzel a rendszer azonnal olvashatóbb lett, mert nemcsak a szöveg, hanem a szín is segített az értelmezésben.

A validáció szintén több rétegű. Vannak frontend oldali ellenőrzések – például ne lehessen üres mezőt elküldeni, vagy logikátlan állapotban továbblépni –, és vannak backend oldali ellenőrzések, például idegen kulcsok vagy egyedi mezők szintjén. A két réteg együtt biztosítja a robusztus működést.

14. Buildelés, Node/npm környezet és hibakezelés

A React-alprojektek használata szükségessé teszi a Node.js és npm környezet megfelelő beállítását. A gyakorlatban ez nem volt teljesen triviális, mert bizonyos Node/npm verziók mellett az install vagy build folyamat hibákat dobhat. A projekt során felmerült tipikus hiba például a 'vite is not recognized' üzenet, amely arra utal, hogy a node modules és a projektfüggőségek még nincsenek telepítve.

Egy másik gyakorlati tanulság az volt, hogy a túl friss Node/npm kombinációk néha hibásabban működnek, mint a stabil LTS környezetek. Emiatt telepítéskor és buildeléskor fontos ellenőrizni a node -v és npm -v kimenetét, szükség esetén cache tisztítást végezni, törölni a node_modules és package-lock.json fájlokat, majd újra telepíteni a függőségeket.

A buildfolyamat után a gyökér HTML oldalak már a dist könyvtárból betöltött assetekkel működnek. Emiatt ha forráskódban történik módosítás, de elmarad az újrabuildelés, a böngészőben látható változat és a forrásállapot eltérhet egymástól. Ez a fejlesztési folyamat egyik legfontosabb operatív tanulsága.

15. GitHub, release és csapatmunka

A projekt GitHubra történő feltöltése nemcsak archiválási szempontból fontos, hanem a fejlesztési folyamat dokumentálása miatt is. A commitokból látszik, hogyan épült fel a rendszer, milyen finomhangolások történtek, és melyik rész mikor érte el a stabil állapotot. A release készítése különösen hasznos, ha a beadáskor vagy értékeléskor egy konkrét verziót kell kijelölni véglegesnek.

Csapatmunkában a GitHub használata még hangsúlyosabb. Egyik fejlesztő létrehozhatja a repository-t, a másik collaboratorként csatlakozhat, és a munka történhet közösen ugyanazon main ágon vagy külön branch-ekben. Ennél a projektnél a fájlszintű felosztás a legpraktikusabb: az egyik fél például a frontend modulokat, a másik a backend és dokumentációs részt gondozhatja.

A merge és pull folyamatok helyes kezelése szintén része a dokumentált munkafolyamatnak. Ha a távoli repository már tartalmaz kezdeti állományokat, akkor a helyi és távoli történetek összefésülése szükségessé válhat. Ez a projektben ténylegesen is előkerült, és jól szemlélteti, hogy a verziókezelés nem pusztán fájlok másolgatása, hanem strukturált együttműködés.

16. Tesztelési forgatókönyvek

A tesztelést modulonként célszerű végezni. A JavaScript CRUD esetében ellenőrizni kell, hogy új rekord hozzáadása után a táblázat frissül-e, szerkesztéskor az űrlap helyesen töltődik-e fel, valamint törlés után eltűnik-e a sor. A React CRUD esetében ugyanez mellett meg kell nézni, hogy a toastok helyes típussal jelennek-e meg, és az állapotváltások után nincs-e inkonzisztencia.

A SPA esetében a legfontosabb forgatókönyvek közé tartozik, hogy választás nélkül ne lehessen ellenőrizni, ellenőrzés nélkül ne lehessen tovább lépni, és a pontszám valóban a helyes válaszok számával nőjön. A kalkulátor esetében ellenőrizni kell, hogy a százalékos arányok helyesen számolódnak-e, és az eredménykártya a megfelelő formázást kapja-e.

A Fetch API és Axios moduloknál minden CRUD műveletet külön végig kell próbálni. Emellett meg kell nézni, hogy a táblázatok a megfelelő formázással jelennek-e meg, a számok jó igazítást kapnak-e, a műveleti gombok elhelyezkedése helyes-e, és a lenyíló mezők valóban a háttéradatokból töltődnek-e fel.

17. Követelmények megfeleltetése

A beadandó értékelhetősége szempontjából fontos, hogy az egyes követelménypontokhoz egyértelműen hozzárendelhető legyen a megvalósított rész. A főoldal az index.html fájlban található, a JavaScript CRUD a javascript.html oldalon, a React CRUD a react.html alatt, a SPA a spa.html fájlban. A szerveroldali adatbázis-kommunikáció a fetchapi.html és axios.html oldalak mögött, az api mappában lévő PHP-végpontokon keresztül történik.

Az OOJS követelmény a külön oojs.html oldalon valósul meg, ahol az objektumorientált JavaScript-megoldás grafikus kártyák létrehozásával szemlélteti az osztályalapú szervezést. A központi konfiguráció az include/config.inc.php fájlban található, amely ugyan formálisan nem része a kiírt menüpontoknak, de a rendszer karbantarthatóságának fontos eleme.

A GitHub, a release és a dokumentáció együtt biztosítja, hogy a projekt ne csak kódszinten legyen kész, hanem átadható, követhető és értelmezhető formában is. Ez a fejezet azért fontos, mert a beadandó akkor erős, ha nemcsak működik, hanem jól kommunikálja is a saját felépítését.

Követelmény	Megvalósítás helye	Rövid értelmezés
Főoldal	index.html	Belépési pont és navigáció.
JavaScript CRUD	javascript.html	Tömbalapú kliensoldali CRUD.
React CRUD	react.html + react-crud	Komponensalapú állapotkezelés.
SPA	spa.html + spa-app	Két miniapp egy oldalon.
Fetch API	fetchapi.html + api/foglalkozasok.php	Backenddel kommunikáló JavaScript modul.
Axios	axios.html + axios-react + api/lakosok.php	React + Axios adatkezelés.
OOJS	oojs.html	Objektumorientált JavaScript grafikus rész.

18. Továbbfejlesztési lehetőségek

A jelenlegi rendszer jól teljesíti a beadandó követelményeit, de több irányban is továbbfejleszthető. Az egyik lehetőség egy közös hitelesítési vagy jogosultságkezelési réteg bevezetése, amely meghatározza, mely felhasználó milyen műveletre jogosult. Ez valós alkalmazási környezetben különösen fontos lenne.

További fejlesztési irány lehet valamilyen grafikon vagy dashboard jellegű nézet létrehozása, amely a lakossági és foglalkoztatási adatokat vizuális diagramokon ábrázolja. Ezzel a rendszer közelebb kerülne egy üzleti intelligencia vagy riportkészítő felülethez. Emellett a jelenlegi játékos SPA modul is bővíthető lenne nehézségi szintekkel, időmérővel vagy kérdésbankkal.

A karbantarthatóságot tovább növelhetné, ha a toastok, űrlapkomponensek és táblázati sorok részletesebben újrafelhasználható közös komponensekbe szerveződnének. Ez főleg akkor lenne indokolt, ha a projektből a jövőben nagyobb rendszer fejlődne ki.

19. Összegzés

A beadandó végső formájában egy összetett, de átgondolt felépítésű webalkalmazás lett. A különböző technológiák nem széteső mintapéldákként, hanem közös témára épülő modulokként jelennek meg. A projekt képes bemutatni a klasszikus JavaScript, a React, a SPA-szemlélet, a PHP backend, a MySQL adatbázis és az OOJS együttműködését.

A fejlesztési folyamat során számos apró, de fontos részletre kellett figyelni: buildelési problémákra, CSS és inline stílusok ütközésére, gombállapotok logikájára, adatbázis-konfiguráció központosítására, valamint a felhasználói visszajelzések minőségére. Ezek a feladatok együtt adták azt a tapasztalatot, amely a projektet valódi beadandóvá emelte.

A dokumentáció célja az volt, hogy ezt a folyamatot ne csak összefoglalja, hanem értelmezze is. Így a rendszer nemcsak működő kódként, hanem fejlesztői gondolatmenetként is bemutatható. Ez hosszabb távon is hasznos, mert megkönnyíti a projekt későbbi bővítését, illetve az értékelő számára is egyértelművé teszi a megvalósítás logikáját.

Beadás előtti ellenőrzőlista

- Minden HTML oldal betöltődik webszerveres környezetből.
- Az adatbázis létrejött, az SQL import hibamentesen lefutott.
- A config.inc.php a megfelelő elérési adatokat tartalmazza.
- A React alprojektek npm install és npm run build után friss dist állományokkal rendelkeznek.
- A Fetch API és Axios modul ténylegesen ment, módosít és töröl az adatbázisban.
- A toastok típusa megfelelő: success zöld, error piros, figyelmeztetés sárga.
- A GitHub repository friss, a release és a dokumentáció elérhető.